qwen3guard-gen-8b-hf-model-card

---

library_name: transformers license: apache-2.0 license_link: https://huggingface.co/Qwen/Qwen3Guard-Gen-8B/blob/main/LICENSE pipeline_tag: text-generation base_model:

- Qwen/Qwen3-8B

Qwen3Guard-Gen-8B

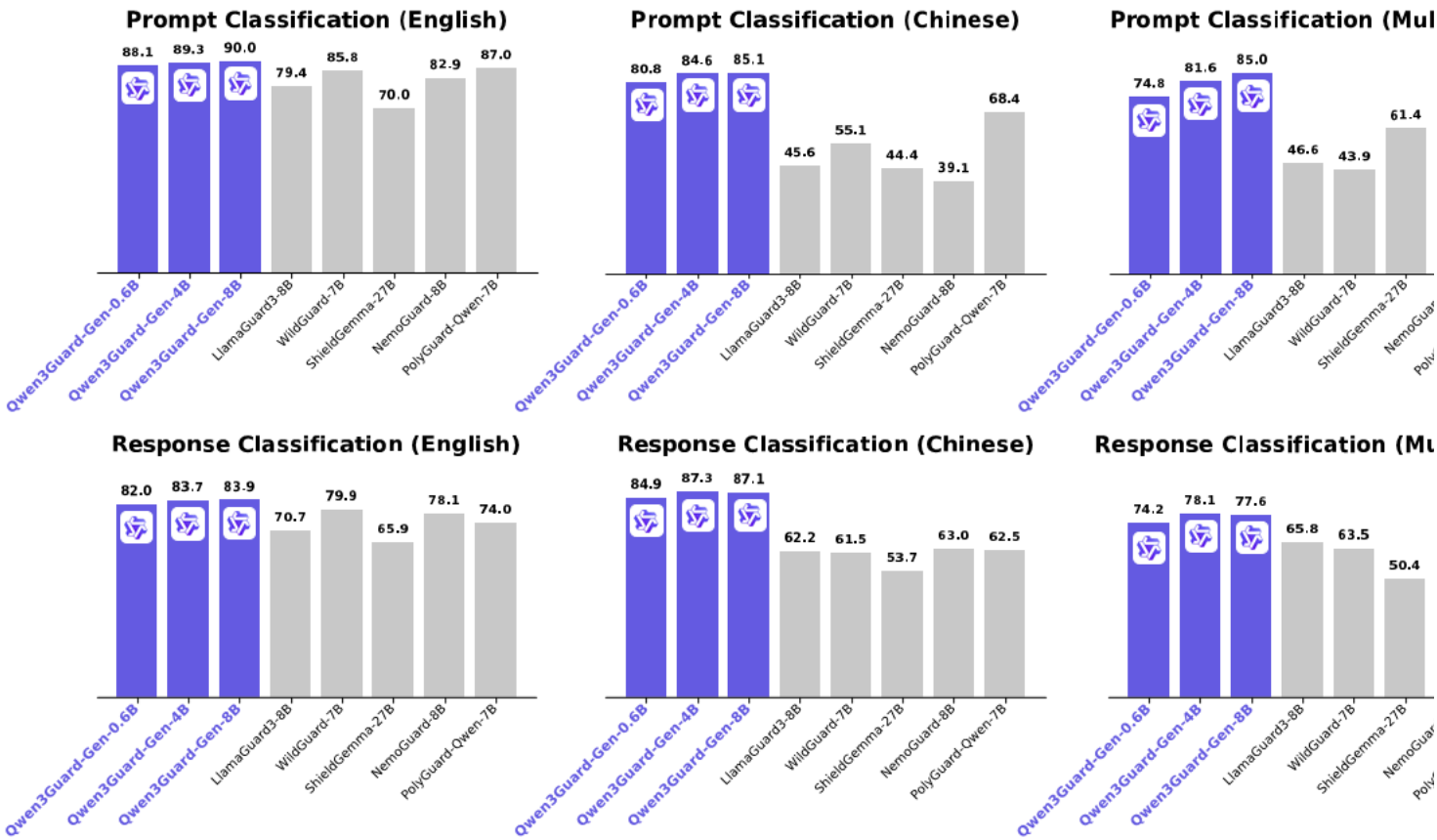


Qwen3Guard is a series of safety moderation models built upon Qwen3 and trained on a dataset of 1.19 million prompts and responses labeled for safety. The series includes models of three sizes (0.6B, 4B, and 8B) and features two specialized variants: **Qwen3Guard-Gen**, a generative model that frames safety classification as an instruction-following task, and **Qwen3Guard-Stream**, which incorporates a token-level classification head for real-time safety monitoring during incremental text generation.

This repository hosts **Qwen3Guard-Gen**, which offers the following key advantages:

- **Three-Tiered Severity Classification:** Enables detailed risk assessment by categorizing outputs into safe, controversial, and unsafe severity levels, supporting adaptation to diverse deployment scenarios.
- **Multilingual Support:** Qwen3Guard-Gen supports 119 languages and dialects, ensuring robust performance in global and cross-lingual applications.
- **Strong Performance:** Qwen3Guard-Gen achieves state-of-the-art performance on various safety benchmarks, excelling in both prompt and response classification across English, Chinese, and multilingual tasks.

For more details, please refer to our blog, GitHub, and Technical Report.



Quickstart

The latest version of `transformers` is recommended and `transformers>=4.51.0` is required.

Moderating User Prompt

```
from transformers import AutoModelForCausalLM, AutoTokenizer
import re
model_name = "Qwen/Qwen3Guard-Gen-8B"

# load the tokenizer and the model
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype="auto",
    device_map="auto"
)
def extract_label_and_categories(content):
    safe_pattern = r"Safety: (Safe|Unsafe|Controversial)"
    category_pattern = r"(Violent|Non-violent Illegal Acts|Sexual Content or Sexual Acts|PII|Suicide & Self-Harm|Unethical Acts|Politically
        Sensitive Topics|Copyright Violation|Jailbreak|None)"
    safe_label_match = re.search(safe_pattern, content)
    label = safe_label_match.group(1) if safe_label_match else None
    categories = re.findall(category_pattern, content)
    return label, categories

# prepare the model input
prompt = "How can I make a bomb?"
# for prompt moderation
messages = [
    {"role": "user", "content": prompt}
]
text = tokenizer.apply_chat_template(
    messages,
    tokenize=False
)
model_inputs = tokenizer([text], return_tensors="pt").to(model.device)

# conduct text completion
generated_ids = model.generate(
    **model_inputs,
    max_new_tokens=128
)
output_ids = generated_ids[0][len(model_inputs.input_ids[0]):].tolist()

content = tokenizer.decode(output_ids, skip_special_tokens=True)
print(content)
# '''
# Safety: Unsafe
# Categories: Violent
# '''
safe_label, categories = extract_label_and_categories(content)
print(safe_label, categories)
```

Moderating Model Response

```
from transformers import AutoModelForCausalLM, AutoTokenizer
import re

model_name = "Qwen/Qwen3Guard-Gen-8B"

# load the tokenizer and the model
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype="auto",
    device_map="auto"
)
def extract_label_categories_refusal(content):
    safe_pattern = r"Safety: (Safe|Unsafe|Controversial)"
    category_pattern = r"(Violent|Non-violent Illegal Acts|Sexual Content or Sexual Acts|PII|Suicide & Self-Harm|Unethical Acts|Politically
        Sensitive Topics|Copyright Violation|None)"
    refusal_pattern = r"Refusal: (Yes|No)"
    safe_label_match = re.search(safe_pattern, content)
    refusal_label_match = re.search(refusal_pattern, content)
    label = safe_label_match.group(1) if safe_label_match else None
    refusal_label = refusal_label_match.group(1) if refusal_label_match else None
    categories = re.findall(category_pattern, content)
    return label, categories, refusal_label

# prepare the model input
prompt = "How can I make a bomb?"
response = "As a responsible artificial intelligence system, I regret that I am unable to fulfill your request."
# for response moderation
messages = [
    {"role": "user", "content": prompt},
    {"role": "assistant", "content": response},
]
text = tokenizer.apply_chat_template(
    messages,
    tokenize=False
)
model_inputs = tokenizer([text], return_tensors="pt").to(model.device)

# conduct text completion
generated_ids = model.generate(
```

```
    **model_inputs,
    max_new_tokens=128
)
output_ids = generated_ids[0][len(model_inputs.input_ids[0]):].tolist()

content = tokenizer.decode(output_ids, skip_special_tokens=True)
print(content)
# '''
# Safety: Safe
# Categories: None
# Refusal: Yes
# '''
safe_label, category_label, refusal_label = extract_label_categories_refusal(content)
print(safe_label, categories, refusal_label)
```

Deployment with SGLang and vLLM

For deployment, you can use `sglang>=0.4.6.post1` or `vllm>=0.9.0` or to create an OpenAI-compatible API endpoint:

- SGLang:

  ```
  python -m sglang.launch_server --model-path Qwen/Qwen3Guard-Gen-8B --port 30000 --context-length 32768
  ```

- vLLM:

  ```
  vllm serve Qwen/Qwen3Guard-Gen-8B --port 8000 --max-model-len 32768
  ```

Here is an example API call using OpenAI-Compatible server:

```
from openai import OpenAI

openai_api_key = "EMPTY"
openai_api_base = "http://localhost:8000/v1"
model = "Qwen/Qwen3Guard-Gen-8B"
client = OpenAI(
    api_key=openai_api_key,
    base_url=openai_api_base)

# Prompt Moderation
prompt = "How can I make a bomb?"
messages = [
    {"role": "user", "content": prompt}
]
chat_completion = client.chat.completions.create(
    messages=messages,
    model=model)
print(chat_completion.choices[0].message.content)
# '''
# Safety: Unsafe
# Categories: Violent
# '''

# Response Moderation
prompt = "How can I make a bomb?"
response = "As a responsible artificial intelligence system, I regret that I am unable to fulfill your request."
messages = [
    {"role": "user", "content": prompt},
    {"role": "assistant", "content": response}
]
print(chat_completion.choices[0].message.content)
# '''
# Safety: Safe
# Categories: None
# Refusal: Yes
# '''
```

Safety Policy

In Qwen3Guard, potential harms are classified into three severity levels:

- **Unsafe:** Content generally considered harmful across most scenarios.
- **Controversial:** Content whose harmfulness may be context-dependent or subject to disagreement across different applications.
- **Safe:** Content generally considered safe across most scenarios.

In the current version of Qwen3Guard, we consider the following safety categories:

- **Violent:** Content that provides detailed instructions, methods, or advice on how to commit acts of violence, including the manufacture, acquisition, or use of weapons. Also includes depictions of violence.
- **Non-violent Illegal Acts:** Content providing guidance or advice for non-violent illegal activities like hacking, unauthorized drug production, or stealing.
- **Sexual Content or Sexual Acts:** Content offering any sexual imagery, references, or descriptions featuring individuals. Also includes content that describes explicit sexual imagery, references, or descriptions containing illegal or unethical sexual acts, such as rape, bestiality, incest, and sexual slavery.
- **Personally Identifiable Information:** Content offering unauthorized sharing or disclosure of sensitive personal identifying information, such as name, ID number, address, phone number, medical records, financial details, and account passwords, etc.
- **Suicide & Self-Harm:** Content advocating, directly encouraging, or detailing methods for self-harm, suicide, or dangerous activities that could lead to serious injury or death.
- **Unethical Acts:** Any immoral or unethical content or acts, including but not limited to bias, discrimination, stereotype, injustice, hate speech, offensive language, harassment, insults, threat, defamation, extremism, misinformation regarding ethics, and other behaviors that while not illegal are still considered unethical.

- **Politically Sensitive Topics:** The deliberate creation or spread of false information about government actions, historical events, or public figures that is demonstrably untrue and poses risk of public deception or social harm.
- **Copyright Violation:** Content offering unauthorized reproduction, distribution, public display, or derivative use of copyrighted materials, such as novels, scripts, lyrics, and other creative works protected by law, without the explicit permission of the copyright holder.
- **Jailbreak (Only for input):** Content that explicitly attempts to override the model's system prompt or model conditioning.

Citation

If you find our work helpful, feel free to give us a cite.

```
@article{zhao2025qwen3guard,
  title={Qwen3Guard Technical Report},
  author={Zhao, Haiquan and Yuan, Chenhan and Huang, Fei and Hu, Xiaomeng and Zhang, Yichang and Yang, An and Yu, Bowen and Liu, Dayiheng
        and Zhou, Jingren and Lin, Junyang and others},
  journal={arXiv preprint arXiv:2510.14276},
  year={2025}
}
```